

Portfolio Building — Studio Course

3rd Semester

Coding Competition

Build a Simple Line Editor in C

Two hours. A team of your own choosing. One working editor — designed on paper before a single line is typed into VS Code.

2 hrs

DURATION

1–3

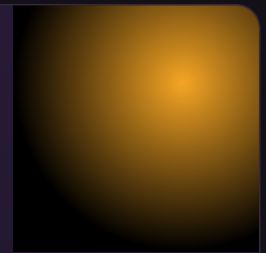
TEAM SIZE

2–3

MIN. FEATURES

100

TOTAL POINTS



•Problem Statement

Read this once as a team before opening any editor.

A **line editor** is a simple text editor that works one line at a time — no cursor moving freely around a 2D screen, just commands that operate on line numbers. Think of it as the ancestor of modern editors: the user types a command, the editor acts on a specific line (or range of lines), and the result is shown back.

Your task: design and build a **working command-line line editor in C** that lets a user create, view, and modify a small text document through simple typed commands — entirely from the terminal, with no GUI.

The editor should hold the document's lines **in memory** using a data structure your team chooses and can justify (array of strings, linked list of lines, dynamic array — your call, but be ready to explain the trade-off), and should support at minimum the features listed below.

•Required & Optional Features

Pick **at least 2–3** from the core list. Anything from the bonus list is extra credit, not a requirement.

CORE • pick 2–3

Insert a line

Add a new line of text at a given line number, shifting existing lines down.

CORE • pick 2–3

Delete a line

Remove the line at a given number, shifting the lines below it up.

CORE • pick 2–3

Display the document

Print all current lines with their line numbers, so the user can see the document state.

CORE · pick 2–3

Save / load a file

Write the in-memory document to a `.txt` file, and read one back in on startup.

BONUS

Search

Find and report which line number(s) contain a given word or phrase.

BONUS

Find & replace

Replace a word or phrase on a given line, or across the whole document.

BONUS

Undo last action

Reverse the most recent insert/delete/edit — the most implementation-heavy bonus, attempt only with time to spare.

BONUS

Line count / word count

Report simple document statistics on request.

•How the Two Hours Should Run

A suggested pace — adjust within your team, but don't skip straight to typing code.

0:00–0:10

Form your team & read the problem

Team of 1–3. Read the full problem statement together before anyone starts thinking about code.

0:10–0:35

Design on paper

Choose your data structure for holding lines, sketch the commands your editor will accept, and outline the functions you'll need. Everyone writes/agrees on the same design.

0:35–1:05

Code the core on paper

Hand-write the key functions (insert, delete, display) before touching VS Code — this is graded, so don't skip it even under time pressure.

1:05–1:40

Build it in VS Code

Type up your paper design, compile with `gcc`, and debug as you go. Test each feature as you add it rather than saving all testing for the end.

1:40–1:55

Write the help file & polish

Document how to use your editor (commands, syntax, examples) in a plain-text or Markdown help file.

1:55–2:00

Push to GitHub

Commit and push the working project — code, help file, and a short README — before time runs out.

•Deliverables

- 1 Paper design** — your data structure choice and hand-written core logic, submitted as-is (photo or scan is fine).
- 2 Working C project** in a GitHub repository — compiles cleanly with `gcc` and runs from the terminal.
- 3 Help file** (`HELP.md` or `help.txt`) explaining each supported command with a usage example.
- 4 README** stating team members, which 2–3+ features were implemented, and how to compile and run the editor.

•Assessment Rubric

100 points total. Every category is graded on the deliverables above, not just the demo.

CRITERION	WHAT'S BEING CHECKED	POINTS
Problem understanding & design	Paper design shows a clear, justified data structure choice and a sensible command set before coding began	15
Core functionality	The 2–3 required features work correctly and handle basic edge cases (e.g., inserting at the end, deleting the only line)	25
Code quality & organization	Readable structure, meaningful variable/function names, no dead or duplicated code, reasonable use of functions	15
Debugging & robustness	Editor doesn't crash on invalid input (bad line numbers, empty document, etc.) and handles simple errors gracefully	10
Bonus features	Any features beyond the required 2–3, implemented correctly	10
Help file quality	Clear, accurate documentation of every supported command with at least one usage example each	10
GitHub repository & README	Clean repo structure, meaningful commit(s), README with team names, features implemented, and run instructions	10
Team collaboration	Evidence (paper design, commit history for multi-member teams) that the work was genuinely shared, not done by one member alone	5
Total		100

